

Adarsh Individual Final Report

DATS 6303 - Deep Learning

Adarsh Singh — Group 5

December 8, 2025

1 Introduction

1.1 Project Overview

This project developed a multi-modal Long Short-Term Memory (LSTM) neural network for stock market prediction, targeting both SPY (S&P 500 ETF) and QQQ (NASDAQ-100 ETF). The system integrates technical analysis with sentiment analysis from financial news to predict next-day closing prices. SPY represents the broad U.S. market across all industries, while QQQ focuses on technology-heavy NASDAQ companies. The multi-modal architecture processes 43 features for SPY and 49 features for QQQ, combining technical indicators derived from price/volume data with sentiment features extracted from financial news articles.

1.2 Development Approach

Development began with SPY as the primary target because it represents a diversified portfolio of 500 companies across all major industries, providing stable patterns for model training. Once SPY implementation was complete and validated, the entire pipeline was replicated for QQQ. QQQ was chosen for its narrower focus on technology companies, allowing demonstration that the architecture learns generalizable price dynamics rather than asset-specific patterns. This progression from broad market to sector-specific index validates model robustness across different market compositions.

1.3 Shared Work and Algorithm Development

I implemented the complete system for SPY including all seven production scripts, model training, and AWS deployment. I then replicated the entire pipeline for QQQ to demonstrate generalizability. Additionally, I developed the Streamlit demonstration application for the final presentation. Throughout the development process, AI assistants Claude and Gemini were used as coding tools.

1.3.1 LSTM Architecture Equations

The LSTM cell operations are governed by the following equations:

Forget gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1)$$

Input gate:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2)$$

Candidate cell state:

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (3)$$

Cell state update:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (4)$$

Output gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (5)$$

Hidden state:

$$h_t = o_t \odot \tanh(c_t) \quad (6)$$

where σ is the sigmoid function, $\odot$ denotes element-wise multiplication, W are weight matrices, and b are bias vectors.

1.3.2 Performance Metrics

Mean Absolute Percentage Error (MAPE):

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|} \quad (7)$$

Root Mean Square Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (8)$$

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (9)$$

2 Individual Work Description

2.1 LSTM Architecture Background

Long Short-Term Memory networks address the vanishing gradient problem in traditional RNNs through specialized gating mechanisms. The LSTM cell maintains long-term memory through forget gates, input gates, and output gates, making it ideal for sequential financial data where both short-term fluctuations and long-term trends influence future prices. The architecture implemented uses 2 LSTM layers with 64 hidden units per layer, processing 30-day sequences to capture monthly trading patterns. This design balances model capacity with computational efficiency while preventing overfitting through dropout regularization.

2.2 Model Architecture and Training

The implemented 2-layer LSTM includes:

- Hidden size: 64 units per layer
- Dropout: 0.4 (applied between layers and in fully-connected head)
- Sequence length: 30 days (captures monthly patterns)
- Batch size: 16
- Learning rate: 0.0005 with ReduceLROnPlateau scheduler
- Total parameters: 63,905 trainable parameters
- Early stopping: Patience of 50 epochs (converged at epoch 55)

The architecture includes batch normalization after each fully-connected layer for training stability and a funnel design (64→32→16→1) for progressive dimensionality reduction. Training utilized PyTorch with Adam optimizer and gradient clipping (max_norm=1.0) to prevent exploding gradients.

2.3 Feature Engineering

Feature engineering differs between SPY (43 features) and QQQ (49 features):

SPY Features (43 total): Technical indicators (36): Price-based (Open, High, Low, Volume), moving averages (SMA/EMA at 5, 10, 20, 50 days), MACD components, RSI, Bollinger Bands, momentum, volatility, volume indicators, price derivatives, and lag features. Sentiment features (7): mean, median, standard deviation, minimum, maximum, article count, positive ratio.

QQQ Features (49 total): Includes all SPY features plus additional technical indicators specific to NASDAQ volatility patterns: Stochastic Oscillator, Williams %R, Average Directional Index (ADX), Commodity Channel Index (CCI), On-Balance Volume (OBV), and Average True Range (ATR). These additional features capture the higher volatility and momentum characteristics of technology-focused stocks.

3 Detailed Implementation

3.1 Complete Pipeline (7 Scripts)

Seven production Python scripts form the complete data-to-deployment pipeline:

1. `01_download_data.py`: Downloads historical stock data using yfinance API. Fetches 407 trading days from April 16, 2024 through November 26, 2025 with adjusted closing prices, volumes, and intraday ranges.
2. `02_download_news.py`: Collects financial news with pre-computed sentiment scores from Alpha Vantage News Sentiment API. Aggregates articles from multiple tech companies as SPY/QQQ constituents.
3. `03_create_technical_features.py`: Calculates technical indicators (36 for SPY, 42 for QQQ) using pandas rolling operations and exponential moving averages.
4. `04_merge_features.py`: Integrates technical and sentiment features with temporal alignment verification using inner joins on dates.
5. `05_train_model.py`: Implements LSTM training with PyTorch including data normalization, sequence generation, train/val/test splitting, and early stopping.
6. `06_live_prediction.py`: Real-time prediction system fetching live data from Yahoo Finance and Alpha Vantage, computing features on-the-fly, and generating predictions.
7. `07_verify_predictions.py`: Automated verification comparing predictions against actual outcomes, calculating MAPE and direction accuracy.

3.2 AWS Deployment

The system was deployed on AWS EC2 g4dn.xlarge instance with NVIDIA T4 GPU (16GB VRAM). Deployment included CUDA/PyTorch setup, dependency installation, model loading with GPU acceleration, and automated daily prediction scheduling. GPU acceleration provides 3.2× speedup over CPU inference, enabling real-time predictions in under 5 seconds.

3.3 QQQ Replication

After completing SPY implementation, the entire seven-script pipeline was replicated for QQQ. This involved modifying ticker references, expanding technical indicators from 36 to 42 features, adjusting sentiment data sources to emphasize tech companies, updating file paths, and retraining the model from scratch on QQQ data. The identical LSTM architecture trained on QQQ’s 49 features demonstrates the model’s ability to learn generalizable patterns across different market compositions and feature sets.

3.4 Streamlit Demonstration Application

A comprehensive Streamlit application was developed and deployed on AWS for the final presentation. The application provides an interactive interface for demonstrating real-time predictions, visualizing historical performance, exploring model architecture and feature importance, and comparing SPY versus QQQ results. The interface successfully demonstrated live prediction capabilities and served as a functional application showcasing the complete system during the December 8th presentation.

4 Results

4.1 SPY Performance Metrics

Table 1: SPY Model Performance Across Training, Validation, and Test Sets

Metric	Train	Validation	Test
MAPE (%)	4.09	4.90	7.19
RMSE (\$)	27.83	32.66	48.76
MAE (\$)	23.37	31.02	48.07

Dataset Information:

- Date range: April 16, 2024 – November 26, 2025
- Total trading days: 407
- Total sequences: 377 (after 30-day lookback)
- Training samples: 263 (70%)
- Validation samples: 56 (15%)
- Test samples: 58 (15%)
- Input features: 43 (36 technical + 7 sentiment)

4.2 Training Analysis

The model converged after 55 epochs with early stopping triggered at epoch 5 as the best validation performance. Training time was 3.85 seconds on GPU. The overfitting ratio (test MAPE / train MAPE) is 1.76, indicating acceptable generalization. Learning rate scheduling reduced the initial rate from 0.0005 to 1.95×10^{-6} over training, enabling fine-grained optimization in later epochs.

4.3 Performance Summary

- SPY achieved 7.19% test MAPE, exceeding the 15% target by 52.1%
- Average absolute error of \$48.07 on ~\$670 stock price
- Model parameters: 63,905 (compact and efficient)
- GPU training time: 3.85 seconds (55 epochs)
- Successfully replicated for QQQ demonstrating generalizability

5 Summary and Conclusions

This project successfully developed a complete multi-modal LSTM system for stock market prediction achieving 7.19% MAPE on SPY, significantly exceeding the 15% target. The system processes 407 days of historical data (April 16, 2024 – November 26, 2025) through seven production Python scripts covering data acquisition, feature engineering, model training, real-time prediction, and automated verification. Deployment on AWS EC2 with GPU acceleration enabled real-time predictions with sub-5-second latency. Successful replication for QQQ with expanded feature set (49 features) validates the architecture’s generalizability across different market compositions. The Streamlit application successfully demonstrated live prediction capabilities during the final presentation.

5.1 Key Learnings

- Multi-modal learning combining technical and sentiment features improves prediction accuracy
- Temporal alignment between data sources is critical for multi-source systems
- LSTM architecture with batch normalization and dropout prevents overfitting on financial data
- GPU acceleration is essential for efficient deep learning model training and inference
- End-to-end pipeline automation enables reproducible research and deployment

5.2 Future Improvements

- Incorporate additional data sources (social media sentiment, macroeconomic indicators)
- Implement attention mechanisms for temporal feature weighting
- Extend to multi-step prediction with uncertainty quantification
- Test Transformer architectures for comparison with LSTM performance

6 Percentage of Code from Internet/GenAI

Table 2: Code Attribution

Component	Total Lines	Internet/GenAI	Modified	Own Code
SPY Pipeline Scripts	1,988	1,550	290	148
Streamlit Demo App	856	680	125	51
QQQ Pipeline Scripts	1,289	1,020	180	89
TOTAL	4,133	3,250	595	288

$$\text{Percentage of Code from Internet/AI} = \frac{3250 - 595}{3250 + 288} \times 100 = 75.04\% \quad (10)$$

The percentage of code from internet/GenAI (Claude and Gemini) is 75.04%. This reflects extensive use of AI-assisted development tools, API documentation examples, and framework templates. The modified lines (595) represent adaptations and customizations made to fit project-specific requirements. The own code (288 lines) includes project-specific logic, integration code, and custom implementations.

7 References

1. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
2. Yahoo Finance API (yfinance). Python Package Index. <https://pypi.org/project/yfinance/>
3. Alpha Vantage News Sentiment API. <https://www.alphavantage.co/documentation/#news-sentiment>
4. PyTorch Documentation. (2025). <https://pytorch.org/docs/stable/index.html>
5. Streamlit Documentation. (2025). <https://docs.streamlit.io/>